



Programación II

GUÍA DE TRABAJOS PRÁCTICOS finde XL:
MODELADO DE TDAS

Profesor: Hernandez Pablo Eduardo

Participantes

- Escribano, Constanza Agustina
- Serra, Sofía
- Viand, Sofía Pilar
- Zaragoza, Marina Luz.

Turno:

Lunes Turno Tarde

Índice

ACTIVIDAD 1	1
ACTIVIDAD 2	
Estrategia 1	2
BLOQUE 1	3
- Historial de navegación	3
- Error de código	4
- Balance de paréntesis	4
- Reversión de strings	4
- Pilas de llamadas (call center)	4
- Navegación de directorios	4
BLOQUE 2	5
- Fila de cajero	5
- Impresora del laboratorio	5
- Guardia del hospital de clínicas (triage)	5
- Examen final (promocionados vs regulares)	5
- Buffet de la facultad	6
- Procesador de tareas (OS scheduler)	6
- Embarque de aerolíneas	6
- Distribución de tickets IT	7
BLOQUE 3	7
- Padrón electoral	8
- Invitados a la fiesta de fin de cursada	8
- Filtro de spam (blacklist)	8
- Tags de un blog de ingeniería	9
- Gestión de legajos	9
- Operación elegir v.s. sacar	9

ACTIVIDAD 1

Sistema de venta de tickets para eventos.

-Utilizamos la fase de contrato (interfaz) de la cola para que no colapse el sistema y que los usuarios puedan acceder a la compra de su entrada sin complicaciones.

-Contrato (Interfaz)

El cliente exige que el sistema tenga estas operaciones:

InicializarCola()

Inicia el sistema de atención con la cola vacía.

Acolar(cliente)

Agrega un nuevo cliente al final de la cola de espera.

Desacolar()

Llama al siguiente cliente para el ingreso a la página web y lo elimina de la cola.

Primero()

Permite ver quién es el próximo cliente para ingresar a la compra de las entradas.

ColaVacia()

Indica si no hay clientes esperando.

-Los usuarios de (alto nivel) → solo utilizan la interfaz. No saben cómo se maneja la información internamente.

-Implementadores → desarrollan la lógica interna del sistema para que la cola funcione correctamente, deben garantizar que el sistema respete FIFO.

ACTIVIDAD 2

Estrategia 1:

Esta estrategia usa una variable externa que almacena el número de elementos y apunta a la primera posición libre.

Es rápida porque no hay que mover elementos. La desventaja es que necesitas una variable extra fuera del arreglo para controlar la cantidad.

Estrategia 2:

Acá el tope de la pila siempre está en arreglo[0]. Cada vez que se apila un elemento hay que desplazar todos los elementos hacia la derecha para dejar libre la posición 0.

La ventaja es que el tope siempre está en el mismo lugar, pero es muy lenta, porque mover elementos cuesta tiempo de CPU .

Estrategia 3:

En esta estrategia la posición 0 del arreglo guarda cuántos elementos hay en la pila. Los datos empiezan desde la posición 1, y el tope está en la última posición ocupada.

Es rápida porque no hay que desplazar elementos ($O(1)$), pero se pierde una posición del arreglo para guardar la cantidad.

Conclusión:

La estrategia 1 y la 3 son las más eficientes porque no mueven elementos. La 2 es la peor en rendimiento ya que cada push requiere desplazar toda la pila. La más común y clara suele ser la estrategia 1.

Clase 2

Guía finde XL

BLOQUE 1

- **Historial de navegación:**

Tomamos el buscador de chrome como elemento [0] y mediante se visitan las páginas las mismas irán tomando la posición [i+1] (fi.uba.ar [1], campus.utn.edu.ar [2] y stackoverflow.com [3]). A la hora de presionar la flecha de “atrás” se regresará a la página cuya posición sea de forma [i-1] hasta llegar al elemento 0 es decir el buscador de chrome.

- **Error de código:**

Se guarda el último estado del “código” y se lo apila completo cada vez que el usuario realiza un cambio.

El estado anterior se recupera desapilado el último elemento de la pila

- **Balance de paréntesis:**

Los paréntesis actúan como divisores de términos y por lo mismo al procesar un término el resultado del mismo se apila. Se apila el paréntesis , el siguiente paréntesis, la variable A y cuando llega al signo + se desapila como Auxiliar(A), L... una vez apilado comparamos el resultado (a+b) para poder así seguir con el siguiente término (*c). cuando se lo haya comparado se desapila a+b para poder apilar el resultado a+b multiplicado por c.

- **Reversión de strings:**

Para poder dar vuelta la palabra “ALGORITMOS” primero hay que InicializarPila(P), luego vamos a recorrer la palabra letra por letra y comenzaremos a apilar, la primera letra [0] va a ser “A”, y al mismo tiempo esta será el tope. Esto va a suceder con todas las letras hasta que lleguemos a la última, que será la letra “S” y quedará como tope, por esto también va a ser la primera que se desapilará cuando PilaVacía(P)=false comenzando a invertir la palabra hasta que demos con el resultado “SOMTIROGLA”.

- **Pilas de llamadas (call center):**

En el momento de la ejecución de la suma, la función que se encuentra en el tope de la pila es sumar (), ya que es la última función que se llama.

Las funciones anteriores quedan en espera hasta que Sumar() finalice y se desapila.

- ***Navegación de directorios:***

Se utiliza a modo de almacenamiento de las rutas anteriores. Cada vez que el usuario ingresa a una carpeta, se apila la carpeta actual. Cuando selecciona “subir nivel”, se desapila la última ruta y se regresa a esa carpeta.

BLOQUE 2:

- ***Fila de cajero:***

Es cola de prioridad, los clientes llegan y se acomodan dependiendo de su “nivel” de prioridad en la cola.

- ***Impresora del laboratorio:***

El orden de salida va a ser el mismo que el orden de entrada, por lo que el primero que sea enviado es el primero que se va a imprimir.

- ***Guardia del hospital de clínicas (triage):***

El nivel de prioridad deberá determinar la urgencia de quien es tratado

- (*prioridad $0 < p < 30$*) Curaciones de heridas leves, control de presión arterial, primeros auxilios básicos, atención de enfermedades respiratorias o gastrointestinales leves. En esta categoría entran los casos que normalmente podrían ser tratados en una salita de primeros auxilios (tiempo de espera 60 mins o más).
- (*prioridad $30 < p < 70$*) Condiciones que requieren atención rápida pero no ponen en riesgo la vida de forma inminente, tales como fiebre alta, fracturas cerradas, dolor abdominal agudo, dolor de oído intenso, cortes que requieren sutura. En esta categoría entran los casos que normalmente podrían ser tratados en una sala de urgencias (tiempo de espera 15 a 60 mins máximo).
- (*prioridad $70 < p < 100$*) Riesgo de vida real, paro cardíaco, accidente cerebrovascular (ACV), traumas graves, dificultad respiratoria severa, hemorragias incontrolables, pérdida de conciencia. En esta categoría entran los casos que deben ser tratados en una sala de emergencias donde la atención debe ser inmediata.

- ***Examen final (promocionados vs regulares):***

Primero sale el promocionado ya que, al utilizar una cola con prioridad, la prioridad prevalece sobre el orden de llegada.

Para el código se utiliza una escala del 1 al 10, donde 10 es la mayor prioridad. En la siguiente demostración, aunque el estudiante regular llegó primero (ej; 8:00a.m), el promocionado (ej;8:30a.m) tiene mayor prioridad, por lo que será el primero en salir.

```
PriorityQueue<Integer> cola = new PriorityQueue<>((a, b) -> b - a);
```

```
cola.add(5); // Regular
```

```
cola.add(10); // Promocionado
```

```
System.out.println(cola.poll());
```

- **Buffet de la facultad:**

Se le asigna un número de prioridad cuyo máximo es la cantidad de sanguchitos de miga que tenga (sanguchitos - 1) y aquel que no tenga dicha prioridad o tenga una prioridad repetida es descartado

- **Procesador de tareas (OS scheduler):**

Se asignan prioridades para indicar qué procesos son más importantes. El scheduler utiliza estas prioridades para decidir cuál ejecutar primero. En situaciones de sobrecarga, los procesos de alta prioridad consumen la mayor parte del CPU, dejando a los de baja prioridad con menos tiempo de ejecución.

Es decir, al ejecutar el código el sistema operativo va a decidir según la prioridad. El hilo con mayor prioridad se ejecuta con más frecuencia. Si hay muchos hilos importantes, los de baja prioridad pueden quedar esperando y ejecutarse muy poco.

Ejemplo básico:

```
Thread sistema = new Thread();  
sistema.setPriority(50);
```

```
Thread spotify = new Thread();  
spotify.setPriority(10);
```

- **Embarque de aerolíneas:**

Si llegan dos personas con la misma prioridad al mismo tiempo, sube el primero que llegara.

- **Distribución de tickets IT:**

El valor de prioridad 999 es designado con ese valor ya que es un error crítico que afecta a todo el sistema por lo cual tiene mayor importancia mientras que el del 0 no tiene tanta relevancia para ser solucionado en el momento ya que no presenta un alto impacto en su funcionamiento.

BLOQUE 3:

- **Padrón electoral:**

```
public boolean pertenece(int x) { 3 usages
    nodo alumnos = c;
    while ((alumnos != null) && (alumnos.info != x)) {
        alumnos = alumnos.sig;
    }
    return (alumnos != null);
}
```

- **Invitados a la fiesta de fin de cursada**

En caso de que el valor ya exista el mismo no se agrega por que pertenece al conjunto (antes de agregar se debe verificar si pertenece).

```
public void agregar(int x) { 3 usages
    if (!this.pertenece(x)) {
        nodo invitados = new nodo();
        invitados.info = x;
        invitados.sig = c;
        c = invitados;
    }
}
```

- **Filtro de spam (blacklist)**

Verificamos si alguna de las palabras del mail pertenece al conjunto palabras prohibidas y se devuelve un valor (true).

```
public boolean Es_Spam(String mail) { no usages
    for (int i = 0; i < palabras.length; i++) {
        if (palabrasSpam.pertenece(palabras[i])) {
            return true;
        }
    }
    return false;
}
```

- **Tags de un blog de ingeniería:**

Utilizamos un conjunto ya que se agruparan los post (lo que en este ejemplo sería el conjunto cuyos elementos son los tags) y se verificarán coincidencias entre elementos de distintos posts.

- **Gestión de legajos:**

Cada carrera representa un conjunto cuyos elementos son los legajos de los alumnos inscriptos, debemos buscar la intersección entre dichos conjuntos (alumnos anotados a ambas carreras).

```
public static void main(String[] args) {  
    int[] Licenciatura_en_informatica = {1001, 1002, 1003};  
    int[] Tecnicatura_en_desarrollo_de_vj = {2001, 1002, 3003};  
  
    boolean legajorepetido = false;  
  
    for (int i = 0; i < Licenciatura_en_informatica.length; i++) {  
        for (int j = 0; j < Tecnicatura_en_desarrollo_de_vj.length; j++) {  
            if (Licenciatura_en_informatica[i] == Tecnicatura_en_desarrollo_de_vj[j]) {  
                System.out.println("Legajo repetido: " + Licenciatura_en_informatica[i]);  
                legajorepetido = true;  
            }  
        }  
    }  
  
    if (!legajorepetido) {  
        System.out.println("No hay alumnos en ambas carreras.");  
    }  
}
```

- **Operación elegir v.s. sacar:**

el elemento que se devuelve tras elegir dos veces seguidas sin sacar va a depender del ... es decir si se elige siempre el primer elemento entonces la devolución será siempre la misma mientras que si dicha aclaración no está los elementos devueltos serán distintos